

Build Plan — MCP Security Gateway

Greenfield rebuild. The existing `gateway/`, `dashboard/`, and `demo/` code is the **reference prototype**: keep it runnable and untouched until Phase 3, because its demo scenarios are the acceptance bar the new build must clear.

How to use this file: phases are strictly ordered unless marked independent. Work top-down; check items off; each phase ends with **exit criteria** that must pass before moving on. Sizes: S $\approx$ one session, M $\approx$ 2–3 sessions, L $\approx$ 4+ sessions.

➔ **You are here: Phase 0, nothing started.**

Phase 0 — Repo scaffold + core proxy MVP (size: M)

The skeleton everything hangs off: new src-layout package, asyncio stdio proxy, pipeline with the minimal stages, file audit.

- [] `pyproject.toml` (uv, extras stubs, ruff/pyright/pytest config), `src/mcp_gateway/` layout per ARCHITECTURE §7
- [] `core/context.py` — `CallContext`, `Principal`, `Decision` dataclasses
- [] `protocol/` — JSON-RPC codec, request-id correlation, passthrough rules
- [] `transports/stdio.py` — asyncio subprocess + stdin/stdout pumps, timeout budgets
- [] `core/pipeline.py` — interceptor chain with stages: Decode → Route → Policy(match only: allow/block/default) → Forward; response Correlate → Deliver
- [] `audit/` — event schema (`schema_version: 1`), recorder, JSONL spool sink
- [] `cli/` — `mcp-gateway wrap --policy ... --audit ... -- <server cmd>`
- [] Tests: codec, pipeline order, allow/block goldens; CI workflow (ruff + pyright + pytest)

Exit criteria: `mcp-gateway wrap` in front of the prototype's `demo/mock_server.py` passes a ported `run_demo` scenarios 1, 4, 5 (passthrough, explicit block, default-deny) with audit events written.

Phase 1 — Policy engine v2 (size: M)

- [] YAML+JSON loader, published JSON Schema (`$schema` autocomplete), `schema_version`
- [] Layered merge (base < connector pack < company override) + precedence tests
- [] Matcher: exact > glob > default; role overlays (field-replace semantics)
- [] Actions as `ActionHandler` plugins: allow, block, redact(stub), rewrite, quarantine, require_approval(stub wired in Phase 3)
- [] Constraints: `Constraint` interface + regex builtin; rewrites (set/append/unless_match)

- [] `tools/list` filtering stage (hide policy-blocked tools from the model)
- [] `mcp-gateway policy validate + policy show --effective`
- [] Golden decision test harness (`policy_tests.yaml` format — reused later by CI/CD phase)

Exit criteria: prototype `policies.json` semantics fully reproduced via a converted pack; golden suite green; `tools/list` visibly filtered.

Phase 2 — Redaction subsystem (size: L — the flagship)

- [] `redaction/engine.py` — detector pipeline, span merge by confidence, operator application, report
- [] `detectors/regex_pii.py` — email/phone/SSN/IP/card **with validators** (Luhn, SSN area rules, octet ranges)
- [] `detectors/secrets.py` — AWS keys, GitHub PATs, Slack tokens, JWTs, private-key blocks, high-entropy strings
- [] `detectors/presidio.py` — optional extra; pool-executor execution; size cap + chunking + latency budget
- [] `detectors/custom.py` — company recognizers from config (employee IDs, hostnames, codenames)
- [] `operators/` — mask, partial-mask, deterministic HMAC hash, tokenize (vault + envelope encryption), drop
- [] `context.py` — allowlists/denylists, context words; `structured.py` — JSON-path targeting, key-name hints
- [] `profiles.py` — secrets-only / standard / strict; policy rules reference profiles
- [] Fail-closed wiring: detector error or over-budget on response path → quarantine result
- [] `tests/redaction_corpus/` — labeled corpus + **precision/recall eval harness** per entity per detector

Exit criteria: corpus eval reports published in CI; a GitHub PAT planted in a mock tool result is caught end-to-end; prototype `redact.py` behavior is a strict subset of the new engine.

Phase 3 — Session controls + approvals + anomaly (size: M)

- [] `state/memory.py` store: session registry, history, taint, risk (interfaces ready for Redis)
- [] Sequence/taint gate stage; risk scoring with thresholds + auto-suspend; suspension broadcast via store
- [] Pipeline ordering per ARCHITECTURE §2: constraints → sequence/taint → **approval last**; taint marking only after gates pass
- [] `approvals/broker.py` (deadline, fail-closed) + console channel (HTTP callback)
- [] `anomaly/` — port heuristic + Claude backends; debounced (not every call); verdict → risk points
- [] Quarantine response path end-to-end

Exit criteria: prototype `attack_scenario.py` and `feature_demo.py` scenarios pass against the new gateway (ported as e2e tests); approval asked only for calls that pass all other gates.

Phase 4 — Console v2 (size: M)

- [] FastAPI app ([server] extra): REST + OpenAPI — sessions, events, policy, approvals, backtest
- [] SQLite (WAL) index store fed from the spool; rebuildable (mcp-gateway audit reindex)
- [] SSE live feed with Last-Event-ID resume; approvals UI; sessions + replay; policy view
- [] mcp-gateway policy backtest --audit <log> in core CLI; console backtest panel calls the same engine
- [] Console authn (session cookie against local users now; OIDC later), read-only vs approver roles

Exit criteria: console_demo flow works browser-first (live feed, click-to-approve, replay); curl against the OpenAPI spec covers every console feature.

Phase 5 — Streamable HTTP transport + central mode (size: L)

- [] transports/streamable_http.py — MCP Streamable HTTP endpoint, Mcp-Session-Id , SSE streams
- [] Multi-upstream routing: /servers/<name>/mcp bound to pack + policy; per-upstream supervision/backoff
- [] state/redis.py + state/postgres.py (index); config switches memory/sqlite ↔ redis/postgres
- [] mcp-gateway serve --config gateway.yaml ; Dockerfile + docker-compose (gateway + console + redis + postgres)
- [] Load test: sustained 100 calls/sec, p99 added latency < 50 ms on regex path

Exit criteria: Claude Code connects to http://gateway/servers/filesystem/mcp and is policed identically to sidecar mode; two replicas share taint/risk via Redis.

Phase 6 — Connector framework + GitHub pack (size: L)

- [] connectors/base.py + registry + mcp-gateway add <name> ; override file mechanism
- [] GitHub pack per ARCHITECTURE §4: full tools.yaml inventory (risk-classified), policy.yaml (reads redacted, CI logs quarantined, writes approval-gated, destructive blocked, default-deny)
- [] Taint model: issue/PR bodies + comments = sources; push/PR/comment/gist = sinks
- [] constraints.py : org allowlist, protected branches; rewrites: draft PRs, search caps
- [] detectors.py : PAT patterns, Actions-log secrets; roles.yaml : developer/reviewer/release-manager/bot
- [] policy_tests.yaml goldens for every rule; pack README with threat model
- [] E2E demo: poisoned-issue → attempted exfil PR, blocked; recorded as acceptance test

Exit criteria: real github-mcp-server policed end-to-end; the pack is the documented template for all future connectors.

Phase 7 — Jira pack (size: M) · Phase 8 — Slack pack (size: M)

- [] Jira: JQL constraint plugin (project allowlist, forbid unbounded sweeps), `maxResults` rewrite, customer-PII redaction profile, JSM content = taint source, roles support-agent/project-admin/bot, goldens
- [] Slack: channel-ACL constraints (block `#exec`, `#hr-*` reads), history redaction (PII+secrets), post = approval to external/shared channels, `rate_limit` action (new ActionHandler), taint private-read → post, goldens

Exit criteria (each): real server policed e2e; pack authored purely with framework primitives — zero engine changes (that's the pluggability proof).

Phases 9–11 are independent of each other; 9 before 11 is recommended (principal-level audit enriches SIEM events).

Phase 9 — Identity: OIDC (size: M)

- [] `identity/oidc.py` — JWT validation, JWKS fetch/cache/rotation, fail-closed expiry policy
- [] `identity/mapping.py` — IdP groups → roles (`identity.yaml`); Okta + Auth0 documented setups
- [] `identity/apikey.py` for headless agents; `principal` on every audit event; console OIDC login

Exit criteria: live Okta dev tenant: two users in different groups get different policy treatment on the same tool; revoked token fails closed.

Phase 10 — Policy CI/CD (size: S–M)

- [] `policy test` (golden harness from Phase 1, CLI-first), `validate`, `backtest` — CI-friendly output
- [] GitHub Action: PR → validate + test + backtest, **diff posted as PR comment**
- [] Merge → versioned bundle (content hash + signature); gateway verifies, atomically swaps, keeps last-known-good

Exit criteria: a policy PR in a demo repo shows the blast-radius comment; a tampered bundle is rejected with an audit event.

Phase 11 — Audit sinks: SIEM (size: M)

- [] Spool-reader sink framework (at-least-once, batch, backoff, watermark alarm)
- [] `sinks/s3.py` — gzip batches, `dt=/hour=` partitioned keys; `sinks/splunk.py` — HEC + retry; `sinks/webhook.py`

- [] `audit/ocsf.py` — OCSF (and ECS) mapping; Splunk dashboard example in docs

Exit criteria: SIEM outage test — sink down for an hour, zero loss, zero hot-path stalls, spool drains on recovery.

Phase 12 — DX & release polish (size: M)

- [] Quickstarts: `pipx install` sidecar in <5 min; `docker compose up` central demo
 - [] Docs site (mkdocs-material): concepts, per-connector threat models, policy reference from JSON Schema
 - [] Prometheus metrics + `/healthz` `/readyz` ; SBOM per release; versioning policy; delete the prototype
-

Dependency graph

```
0 → 1 → 2 → 3 → 4 → 5 → 6 → 7/8 (parallel)
                    5 → 9, 10, 11 (independent; 9 before 11 recommended) → 12
```

Phases 1–3 give a sidecar strictly better than the prototype. Phase 5 unlocks enterprise. Phase 6 sets the connector template that makes 7/8 fast.